



(12) 发明专利申请

(10) 申请公布号 CN 121283943 A

(43) 申请公布日 2026. 01. 06

(21) 申请号 202511145693.8

(22) 申请日 2025.08.15

(71) 申请人 北京大学

地址 100871 北京市海淀区颐和园路5号

(72) 发明人 杨全 刘子瑞 樊卓宸

(74) 专利代理机构 北京路浩知识产权代理有限公司 11002

专利代理师 蔺向星

(51) Int. Cl.

H04L 45/7453 (2022.01)

H04L 45/74 (2022.01)

H04B 7/185 (2006.01)

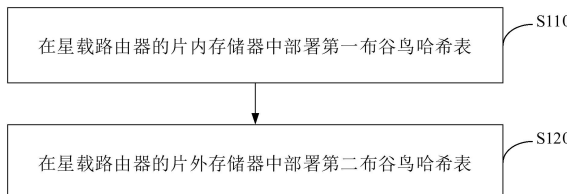
权利要求书2页 说明书12页 附图3页

(54) 发明名称

星载路由器键值存储的片外一次访存管理方法、装置

(57) 摘要

本发明涉及数据管理领域,提供了一种星载路由器键值存储的片外一次访存管理方法、装置,该方法包括:在星载路由器的片内存储器中部署第一布谷鸟哈希表;在星载路由器的片外存储器中部署第二布谷鸟哈希表;其中,所述第一布谷鸟哈希表用于存储键的指纹;所述第二布谷鸟哈希表用于存储完整的键值对;所述第一布谷鸟哈希表中的每个指纹与所述第二布谷鸟哈希表中同一索引位置的键值对一一对应。本发明解决了现有技术中高负载键值存储与低片外访存次数不可兼得的问题,实现了在保持高负载率的同时确保每次键值操作仅需一次片外存储器访问。



1. 一种星载路由器键值存储的片外一次访存管理方法,其特征在于,包括:
在星载路由器的片内存储器中部署第一布谷鸟哈希表;
在星载路由器的片外存储器中部署第二布谷鸟哈希表;
其中,所述第一布谷鸟哈希表用于存储键的指纹;所述第二布谷鸟哈希表用于存储完整的键值对;所述第一布谷鸟哈希表中的每个指纹与所述第二布谷鸟哈希表中同一索引位置的键值对一一对应。

2. 根据权利要求1所述的星载路由器键值存储的片外一次访存管理方法,其特征在于,所述第一布谷鸟哈希表和所述第二布谷鸟哈希表均包括两个候选桶;所述候选桶为数组集合,每个数组集合中包括多个数组;每个数组中包括多个数组元素槽位;

通过两个索引哈希函数对键或键的指纹在候选桶中的存储位置进行定位。

3. 根据权利要求2所述的星载路由器键值存储的片外一次访存管理方法,其特征在于,所述通过两个索引哈希函数对键或键的指纹在候选桶中的存储位置进行定位,具体包括:

基于第一索引哈希函数,根据数组集合中的数组个数确定第一索引;

根据所述第一索引确定键或键的指纹在第一候选桶中的存储位置;

基于第二索引哈希函数,根据所述第一索引、键的指纹和数组集合中的数组个数确定键或键的指纹在第二候选桶中的存储位置。

4. 根据权利要求3所述的星载路由器键值存储的片外一次访存管理方法,其特征在于,当在所述第二布谷鸟哈希表中查找键值对时,所述方法还包括:

计算待查找键值对在候选桶中的索引;

在片内存储器的两个候选桶中确定与所述待查找键值对在候选桶中的索引对应的存储位置是否存在与所述待查找键值对匹配的指纹;

确定与所述待查找键值对匹配的指纹存在,访问片外存储器的对应存储位置,查找键值对;或者

确定与所述待查找键值对匹配的指纹不存在,返回键不存在的通知。

5. 根据权利要求3所述的星载路由器键值存储的片外一次访存管理方法,其特征在于,当在所述第二布谷鸟哈希表中插入键值对时,所述方法还包括:

计算待插入键值对在候选桶中的索引;

检查所述片内存储器中与所述待插入键值对在候选桶中的索引对应的存储槽位是否为空;

若两个候选桶中的任一候选桶对应的存储槽位为空,则将所述待插入键值对的指纹存储至空槽,并将所述键值对存储至所述片外存储器中对应索引的槽位;

若两个候选桶对应的存储槽位均非空槽,则通过广度优先搜索重新定位槽位。

6. 根据权利要求5所述的星载路由器键值存储的片外一次访存管理方法,其特征在于,所述通过广度优先搜索重新定位槽位,具体包括:

初始化搜索队列,通过广度优先搜索确定最短踢出路径;

将待插入键值对和所述最短踢出路径上的片外键值对重新定位至各自新的槽位。

7. 根据权利要求3所述的星载路由器键值存储的片外一次访存管理方法,其特征在于,当删除所述键值对时,所述方法还包括:

计算待删除键值对在候选桶中的索引;

在片内存储器的两个候选桶中确定与所述待删除键值对在候选桶中的索引对应的存储位置是否存在与所述待删除键值对匹配的指纹；

确定与所述待删除键值对匹配的指纹存在,检查与所述待删除键值对匹配的指纹所在的槽位是否有锁；

确定没有锁,从所述第一布谷鸟哈希表中删除与所述待删除键值对匹配的指纹。

8.根据权利要求3所述的星载路由器键值存储的片外一次访存管理方法,其特征在于,当更新所述键值对时,具体包括:

计算待更新键值对在候选桶中的位置；

在片内存储器的两个候选桶中确定与所述待更新键值对在候选桶中的索引对应的存储位置是否存在与所述待更新键值对匹配的指纹；

确定与所述待更新键值对匹配的指纹存在,检查与所述待更新键值对匹配的指纹所在的槽位是否有锁；

确定没有锁,向与所述待更新键值对匹配的指纹所在的槽位添加写锁,然后通过写请求更新片内存储器和片外存储器中对应的槽位的值。

9.根据权利要求3所述的星载路由器键值存储的片外一次访存管理方法,其特征在于,动态扩展所述候选桶,具体包括:

将第一候选桶和第二候选桶按整数倍复制扩展；

根据扩展倍数更新扩展候选桶的索引哈希函数；

根据所述扩展候选桶的索引哈希函数清理扩展候选桶中的冗余元素；

其中,清理冗余元素的模式包括主动模式和懒模式。

10.根据权利要求9所述的星载路由器键值存储的片外一次访存管理方法,其特征在于,通过主动模式清理冗余元素,具体包括:在复制扩展候选桶后立即对所有扩展候选桶中的冗余元素进行删除；

通过懒模式清理冗余元素,具体包括:

在复制扩展候选桶后,为每个扩展候选桶设置待清理标记；

当首次访问带标记的扩展候选桶时,将带标记的扩展候选桶中的所有冗余元素清除；

将清楚冗余元素后的扩展候选桶的待清理标记删除。

11.一种星载路由器键值存储的片外一次访存管理装置,其特征在于,包括:

第一部署模块,用于在星载路由器的片内存储器中部署第一布谷鸟哈希表；

第二部署模块,用于在星载路由器的片外存储器中部署第二布谷鸟哈希表；

其中,所述第一布谷鸟哈希表用于存储键的指纹;所述第二布谷鸟哈希表用于存储完整的键值对;所述第一布谷鸟哈希表中的每个指纹与所述第二布谷鸟哈希表中同一索引位置的键值对一一对应。

12.一种电子设备,包括存储器、处理器及存储在所述存储器上并可在所述处理器上运行的计算机程序,其特征在于,所述处理器执行所述计算机程序时实现如权利要求1至10任一项所述星载路由器键值存储的片外一次访存管理方法。

13.一种非暂态计算机可读存储介质,其上存储有计算机程序,其特征在于,所述计算机程序被处理器执行时实现如权利要求1至10任一项所述星载路由器键值存储的片外一次访存管理方法。

星载路由器键值存储的片外一次访存管理方法、装置

技术领域

[0001] 本发明涉及数据管理领域,尤其涉及一种星载路由器键值存储的片外一次访存管理方法、装置。

背景技术

[0002] 随着星载互联网和在轨分布式计算的快速发展,现代星载路由器需要处理日益复杂的链路调度和海量数据转发任务。在这种高动态网络环境下,路由器内部的流表系统面临着高频度、高并发的键值对 (Key-Value, KV) 访问压力,每秒需要处理数百万次的流表查询、插入和更新操作。然而,星载平台具有严格的资源约束条件:处理器的计算能力有限,内存容量显著小于地面设备,能源供应完全依赖太阳能且功耗预算极为紧张。这些限制使得传统的地面网络设备优化方案难以直接适用于太空环境,特别是在片外存储器访问方面,频繁的DRAM读写不仅会产生高达地面设备数十倍的通信延迟,还会急剧增加系统能耗,严重制约星载路由器的整体性能。

[0003] 当前主流的键值存储系统主要采用基于动态完美哈希 (Dynamic Perfect Hashing, DPH) 或布谷鸟哈希 (Cuckoo Hashing) 的索引结构。动态完美哈希虽然能实现常数级查找延迟,但为了保证极低的冲突率,其负载因子通常被限制,导致大量的存储空间被浪费,且在键集合变化时需要频繁重构哈希函数,产生巨大的计算开销。近年来出现的 MapEmbed、TEA 等高负载方案通过多路哈希和桶化结构提升负载因子,但这些方案需要同时检查多个候选桶及其内部多个槽位,使得每次查找操作可能触发多次片外存储器访问,在星载环境下会造成难以接受的延迟和能耗。Bloom Filter 等存在性过滤器虽然能辅助减少无效访问,但其固有的假阳性特性使得系统仍需进行冗余的片外验证,特别是在恶意攻击导致的高比例非法请求场景下,性能会出现显著劣化。

发明内容

[0004] 本发明提供一种星载路由器键值存储的片外一次访存管理方法、装置,解决了现有技术中高负载键值存储与低片外访存次数不可兼得的问题,实现了在保持高负载率的同时确保每次键值操作仅需一次片外存储器访问。

[0005] 本发明提供一种星载路由器键值存储的片外一次访存管理方法,包括如下步骤:

在星载路由器的片内存储器中部署第一布谷鸟哈希表;

在星载路由器的片外存储器中部署第二布谷鸟哈希表;

其中,所述第一布谷鸟哈希表用于存储键的指纹;所述第二布谷鸟哈希表用于存储完整的键值对;所述第一布谷鸟哈希表中的每个指纹与所述第二布谷鸟哈希表中同一索引位置的键值对一一对应。

[0006] 根据本发明提供的一种星载路由器键值存储的片外一次访存管理方法,所述第一布谷鸟哈希表和所述第二布谷鸟哈希表均包括两个候选桶;所述候选桶为数组集合,每个数组集合中包括多个数组;每个数组中包括多个数组元素槽位;通过两个索引哈希函数对

键或键的指纹在候选桶中的存储位置进行定位。

[0007] 根据本发明提供一种星载路由器键值存储的片外一次访存管理方法,所述通过两个索引哈希函数对键或键的指纹在候选桶中的存储位置进行定位,具体包括:基于第一索引哈希函数,根据数组集合中的数组个数确定第一索引;根据所述第一索引确定键或键的指纹在第一候选桶中的存储位置;基于第二索引哈希函数,根据所述第一索引、键的指纹和数组集合中的数组个数确定键或键的指纹在第二候选桶中的存储位置。

[0008] 根据本发明提供一种星载路由器键值存储的片外一次访存管理方法,当在所述第二布谷鸟哈希表中查找键值对时,所述方法还包括:计算待查找键值对在候选桶中的索引;在片内存储器的两个候选桶中确定与所述待查找键值对在候选桶中的索引对应的存储位置是否存在与所述待查找键值对匹配的指纹;确定与所述待查找键值对匹配的指纹存在,访问片外存储器的对应存储位置,查找键值对;或者确定与所述待查找键值对匹配的指纹不存在,返回键不存在的通知。

[0009] 根据本发明提供一种星载路由器键值存储的片外一次访存管理方法,当在所述第二布谷鸟哈希表中插入键值对时,所述方法还包括:计算待插入键值对在候选桶中的索引;检查所述片内存储器中与所述待插入键值对在候选桶中的索引对应的存储槽位是否为空;若两个候选桶中的任一候选桶对应的存储槽位为空,则将所述待插入键值对的指纹存储至空槽,并将所述键值对存储至所述片外存储器中对应索引的槽位;若两个候选桶对应的存储槽位均非空槽,则通过广度优先搜索重新定位槽位。

[0010] 根据本发明提供一种星载路由器键值存储的片外一次访存管理方法,所述通过广度优先搜索重新定位槽位,具体包括:初始化搜索队列,通过广度优先搜索确定最短踢出路径;将待插入键值对和所述最短踢出路径上的片外键值对重新定位至各自新的槽位。

[0011] 根据本发明提供一种星载路由器键值存储的片外一次访存管理方法,当删除所述键值对时,所述方法还包括:计算待删除键值对在候选桶中的索引;在片内存储器的两个候选桶中确定与所述待删除键值对在候选桶中的索引对应的存储位置是否存在与所述待删除键值对匹配的指纹;确定与所述待删除键值对匹配的指纹存在,检查与所述待删除键值对匹配的指纹所在的槽位是否有锁;确定没有锁,从所述第一布谷鸟哈希表中删除与所述待删除键值对匹配的指纹。

[0012] 根据本发明提供一种星载路由器键值存储的片外一次访存管理方法,当更新所述键值对时,具体包括:计算待更新键值对在候选桶中的位置;在片内存储器的两个候选桶中确定与所述待更新键值对在候选桶中的索引对应的存储位置是否存在与所述待更新键值对匹配的指纹;确定与所述待更新键值对匹配的指纹存在,检查与所述待更新键值对匹配的指纹所在的槽位是否有锁;确定没有锁,向与所述待更新键值对匹配的指纹所在的槽位添加写锁,然后通过写请求更新片内存储器和片外存储器中对应的槽位的值。

[0013] 根据本发明提供一种星载路由器键值存储的片外一次访存管理方法,动态扩展所述候选桶,具体包括:将第一候选桶和第二候选桶按整数倍复制扩展;根据扩展倍数更新扩展候选桶的索引哈希函数;根据所述扩展候选桶的索引哈希函数清理扩展候选桶中的冗余元素;其中,清理冗余元素的模式包括主动模式和懒模式。

[0014] 根据本发明提供一种星载路由器键值存储的片外一次访存管理方法,通过主动模式清理冗余元素,具体包括:在复制扩展候选桶后立即对所有扩展候选桶中的冗余元素

进行删除;通过懒模式清理冗余元素,具体包括:在复制扩展候选桶后,为每个扩展候选桶设置待清理标记;当首次访问带标记的扩展候选桶时,将带标记的扩展候选桶中的所有冗余元素清除;将清楚冗余元素后的扩展候选桶的待清理标记删除。

[0015] 本发明还提供一种星载路由器键值存储的片外一次访存管理装置,包括如下模块:

第一部署模块,用于在星载路由器的片内存储器中部署第一布谷鸟哈希表;

第二部署模块,用于在星载路由器的片外存储器中部署第二布谷鸟哈希表;

其中,所述第一布谷鸟哈希表用于存储键的指纹;所述第二布谷鸟哈希表用于存储完整的键值对;所述第一布谷鸟哈希表中的每个指纹与所述第二布谷鸟哈希表中同一索引位置的键值对一一对应。

[0016] 本发明还提供一种电子设备,包括存储器、处理器及存储在存储器上并可在处理器上运行的计算机程序,所述处理器执行所述计算机程序时实现如上述任一种所述星载路由器键值存储的片外一次访存管理方法。

[0017] 本发明还提供一种非暂态计算机可读存储介质,其上存储有计算机程序,该计算机程序被处理器执行时实现如上述任一种所述星载路由器键值存储的片外一次访存管理方法。

[0018] 本发明还提供一种计算机程序产品,包括计算机程序,所述计算机程序被处理器执行时实现如上述任一种所述星载路由器键值存储的片外一次访存管理方法。

[0019] 本发明提供的一种星载路由器键值存储的片外一次访存管理方法、装置,包括以下有益效果:通过在片内SRAM中部署的指纹表仅存储键的压缩特征,而片外DRAM中则保存完整的键值数据。这种片内指纹表的轻量化特性使得所有键的存在性判断都能在快速存储器中完成,有效过滤了无效访问请求;其次,指纹与键值对之间严格的索引位置对应关系,确保每次有效操作都能直接定位到片外存储器的精确位置。这种机制从根本上消除了现有技术需要多次探测候选位置的问题,使得在高负载条件下仍能保证每次键值操作仅需一次片外访问,相比传统方案的多次访问显著降低了通信延迟和能耗。同时,双布谷鸟哈希表的协同设计既继承了布谷鸟哈希高空间利用率的优势,又通过片内精准索引克服了其访存次数多的缺陷,完美适配了星载平台资源受限的应用场景。

附图说明

[0020] 为了更清楚地说明本发明或现有技术中的技术方案,下面将对实施例或现有技术描述中所需要使用的附图作一简单地介绍,显而易见地,下面描述中的附图是本发明的一些实施例,对于本领域普通技术人员来讲,在不付出创造性劳动的前提下,还可以根据这些附图获得其他的附图。

[0021] 图1是本发明提供的星载路由器键值存储的片外一次访存管理方法的流程示意图。

[0022] 图2是本发明提供的将元素直接插入CuckooDuo流表的示例图。

[0023] 图3是本发明提供的涉及BFS操作的元素插入CuckooDuo的示例图。

[0024] 图4是本发明提供的双指纹优化处理指纹冲突的示例图。

[0025] 图5是本发明提供的动态扩展中的基本扩展方法的示例图。

[0026] 图6是本发明提供的星载路由器键值存储的片外一次访存管理装置的结构示意图。

[0027] 图7是本发明提供的电子设备的结构示意图。

具体实施方式

[0028] 为使本发明的目的、技术方案和优点更加清楚,下面将结合本发明中的附图,对本发明中的技术方案进行清楚、完整地描述,显然,所描述的实施例是本发明一部分实施例,而不是全部的实施例。基于本发明中的实施例,本领域普通技术人员在没有作出创造性劳动前提下所获得的所有其他实施例,都属于本发明保护的范围。

[0029] 下面对本发明涉及到的术语进行解释说明。

[0030] 星载路由器(Spaceborne Router),指部署在卫星等航天器上的网络路由设备,用于太空环境下的数据包转发和链路管理。

[0031] 键值对(Key-Value Pair,KV),一种数据存储结构,由唯一标识的键和对应的值组成。

[0032] 动态完美哈希(Dynamic Perfect Hashing,DPH),一种支持动态更新的哈希技术,可确保无冲突的键值映射。

[0033] 布谷鸟哈希(Cuckoo Hashing),一种使用多个哈希函数的开放寻址哈希方案,通过踢出机制解决冲突。

[0034] 静态随机存取存储器(Static Random-Access Memory,SRAM),一种高速缓存存储器,不需要刷新电路即可保持数据。

[0035] 动态随机存取存储器(Dynamic Random-Access Memory,DRAM),需要周期性刷新的主存储器,具有较高存储密度。

[0036] 远程直接内存访问(Remote Direct Memory Access,RDMA),一种网络技术,允许计算机直接访问另一台计算机的内存而无需处理器介入。

[0037] 往返时间(Round-Trip Time,RTT),数据包从发送到接收确认所需的总时间。

[0038] 广度优先搜索(Breadth-First Search,BFS),一种图形搜索算法,按层次遍历节点。

[0039] 指纹(Fingerprint,FP),通过哈希函数生成的键的特征摘要,用于快速比对。

[0040] 模运算(Modular Arithmetic),一种整数算术运算,结果是被除数除以除数后的余数。

[0041] 负载因子(Load Factor),哈希表中已占用槽位与总槽位的比率。

[0042] 假阳性率(False Positive Rate),错误判断为存在的概率。

[0043] 非易失性内存主机控制器接口规范(Non-Volatile Memory Express over Fabrics,NVMe over Fabrics),一种扩展的NVMe协议,支持通过网络访问存储设备。

[0044] 互斥锁(Mutual Exclusion Lock,Mutex),一种同步机制,用于防止多个线程同时访问共享资源。

[0045] 本发明提出一种“片内-片外协同”的高效键值索引机制,适用于星载路由器等高速网络节点设备,在保持极低访存开销的前提下,实现路由器流表高负载、高并发的KV管理操作。为了克服现有的哈希表和动态完美哈希变体的性能问题,本发明提供一种新颖的可

扩展动态完美哈希索引,并基于该动态完美哈希索引设计一套新颖的星载路由器键值存储流表系统,具有关联性=1、装载率高、处理速度极快、无假阳率、带宽开销极低、支持动态扩容等优势,可有效提高存取效率并减少片外访问次数。

[0046] 本发明提出的星载路由器流表系统(命名为CuckooDuo)采用双布谷鸟哈希表协同架构。该系统在星载路由器的片外DRAM存储器中部署第二布谷鸟哈希表CuckooVault布谷鸟哈希表用于存储完整的键值对数据,同时在片内SRAM存储器中部署相同规模的第一布谷鸟哈希表CuckooIndex布谷鸟哈希表用于存储键的指纹信息。CuckooIndex中的每个指纹与CuckooVault中对应位置的键值对保持严格的一一映射关系。在查找操作时,首先将查询键映射至CuckooIndex的两个候选存储桶中进行指纹比对;若在任一候选桶中发现匹配指纹,则从CuckooVault的对应位置检索键值对;若两个候选桶均无匹配指纹,则可判定该键不存在于系统中。

[0047] 下面结合图1-图7具体描述本发明的实施例。

[0048] 图1是本发明提供的星载路由器键值存储的片外一次访存管理方法的流程示意图,如图1所示,该方法包括如下步骤:

S110、在星载路由器的片内存储器中部署第一布谷鸟哈希表。

[0049] S120、在星载路由器的片外存储器中部署第二布谷鸟哈希表。

[0050] 其中,所述第一布谷鸟哈希表用于存储键的指纹;所述第二布谷鸟哈希表用于存储完整的键值对;所述第一布谷鸟哈希表中的每个指纹与所述第二布谷鸟哈希表中同一索引位置的键值对一一对应。

[0051] 根据本发明提供的一种星载路由器键值存储的片外一次访存管理方法,所述第一布谷鸟哈希表和所述第二布谷鸟哈希表均包括两个候选桶;所述候选桶为数组集合,每个数组集合中包括多个数组;每个数组中包括多个数组元素槽位;通过两个索引哈希函数对键或键的指纹在候选桶中的存储位置进行定位。

[0052] 根据本发明提供的一种星载路由器键值存储的片外一次访存管理方法,所述通过两个索引哈希函数对键或键的指纹在候选桶中的存储位置进行定位,具体包括:基于第一索引哈希函数,根据数组集合中的数组个数确定第一索引;根据所述第一索引确定键或键的指纹在第一候选桶中的存储位置;基于第二索引哈希函数,根据所述第一索引、键的指纹和数组集合中的数组个数确定键或键的指纹在第二候选桶中的存储位置。

[0053] 具体地,在星载路由器流表设计中,其片外DRAM大型键值表CuckooVault用来存储海量的键值对(流ID-流信息 元素),被组织为两个桶数组 B_1 和 B_2 ,每个桶数组有 m 个桶,而每个桶有 d 个槽;星载路由器片内SRAM小型索引表CuckooIndex用来存储插入键的指纹(哈希值),其中指纹是使用哈希函数 $FP(.)$ 计算的,也被组织为两个桶数组 I_1 和 I_2 ,其大小与CuckooVault的相同。

[0054] CuckooIndex中的每个指纹都与CuckooVault中的一个键值对具有一一对应的关系。对于每个键,使用两个哈希函数 $h_1(.)$ 和 $h_2(.)$ 将其映射为CuckooVault或CuckooIndex的两个候选桶的索引。由于CuckooIndex仅存储指纹,不能直接使用完整键的哈希值来定位候选桶。经典方案CuckooFilter使用基于指纹的异或值来计算候选桶的索引,但异或操作限制了表大小必须是2的幂,意味着需要分配比实际存储需求更多的存储空

间来满足这一约束,导致空间膨胀、存储效率低。为了打破该限制,本发明提出使用模加法/减法运算来代替异或运算。对于每个key(键),计算其两个候选桶的索引为

$$h_1(\text{key}) = H(\text{key})\%m$$

$$h_2(\text{key}) = (h_1(\text{key}) + \text{hash}(\text{FP}(\text{key})))\%m$$

其中 $H(\text{key})$ 是一个哈希值,并且有 $h_1(\text{key}) = (h_2(\text{key}) - \text{hash}(\text{FP}(\text{key})))\%m$ 。

[0055] 在实现中,CuckooIndex和CuckooVault分别在星载路由器的片内SRAM和片外DRAM部署,并使用单边读取(READ)和写入(WRITE)操作来访问片外DRAM内存。

[0056] 根据本发明提供的一种星载路由器键值存储的片外一次访存管理方法,当在所述第二布谷鸟哈希表中插入键值对时,所述方法还包括:计算待插入键值对在候选桶中的索引;检查所述片内存储器中与所述待插入键值对在候选桶中的索引对应的存储槽位是否为空;若两个候选桶中的任一候选桶对应的存储槽位为空,则将所述待插入键值对的指纹存储至空槽,并将所述键值对存储至所述片外存储器中对应索引的槽位;若两个候选桶对应的存储槽位均非空槽,则通过广度优先搜索重新定位槽位。

[0057] 根据本发明提供的一种星载路由器键值存储的片外一次访存管理方法,所述通过广度优先搜索重新定位槽位,具体包括:初始化搜索队列,通过广度优先搜索确定最短踢出路径;将待插入键值对和所述最短踢出路径上的片外键值对重新定位至各自新的槽位。

[0058] 具体地,为了插入一个元素(新流ID) $e = \langle \text{key}, \text{value} \rangle$,假设key在片外CuckooVault中不存在,其关键思想是使用广度优先搜索(BFS)在本地CuckooIndex中找到最短的踢出路径,然后要么直接将e写入一个空槽(1个往返时间RTT),要么同时读取踢出路径上的所有元素,并将它们与e一起写入新位置(2个RTT)。

[0059] 首先计算为 $h_1(\text{key}) = H(\text{key})\%m$ 和 $h_2(\text{key}) = (h_1(\text{key}) + \text{hash}(\text{FP}(\text{key})))\%m$,以此在片内CuckooIndex中定位候选桶 $I_1[h_1(\text{key})]$ 和 $I_2[h_2(\text{key})]$ 。接着检查片内 $I_1[h_1(\text{key})]$ 或 $I_2[h_2(\text{key})]$ 中是否有空槽。若有,则将元素插入该槽,具体做法是将 $\text{FP}(\text{key})$ 写入CuckooIndex中的槽位,并将e写入CuckooVault中的相应槽位,然后返回插入成功。如果两个候选桶都已满,则开始BFS过程以找到最短的踢出路径。首先初始化一个空队列Q,并将 $I_1[h_1(\text{key})]$ 和 $I_2[h_2(\text{key})]$ 中的所有指纹入队。然后,反复从队列前端弹出指纹,直到踢出路径长度达到预定义的阈值L。对于每个弹出的指纹f,检查它是否可以插入到其备用候选桶中。先计算其备用候选桶的索引。例如,如果f存储在第一个桶数组中,则计算 $h_2(f) = (h_1(f) + \text{hash}(f))\%m$,其中 $h_1(f)$ 为当前桶索引。然后检查备用候选桶 $I_2[h_2(f)]$:

1) 若 $I_2[h_2(f)]$ 有空槽,则BFS已找到最短的踢出路径之一。接下来对片外CuckooVault执行插入操作,首先使用一批READ请求读取踢出路径上的所有元素,然后使用另一批WRITE请求将这些元素(包括e)写入它们的目标槽位。最后,更新CuckooIndex中的指纹,以确保其与CuckooVault的一一映射关系,并返回插入成功。

[0060] 2) 若 $I_2[h_2(f)]$ 也已满,则继续BFS。具体做法是将 $I_2[h_2(f)]$ 中的每个指纹f'及其相应的踢出路径(如果路径长度小于预定义的最大踢出长度L)入队到队列末尾。此过程重

复,直到队列为空,此时返回插入失败。

[0061] 为了应对星载路由器流表处于高装载率时可能带来的插入失败的特殊情况,本发明在其片内SRAM构建一个小型备用存储区stash(大小为8~32个槽位)来存储插入失败的元素。这个stash可视为片内 CuckooIndex 的一部分。查找key时,首先在stash中查找。若key在stash中,则直接返回其值,否则,在候选桶中查找其指纹。

[0062] 本发明提出的双指纹优化来有效地处理指纹碰撞。其关键思想是为具有指纹冲突的元素更改另一个指纹哈希函数,从而给冲突元素第二次插入机会。当通过更改指纹无法解决冲突时,将冲突元素插入stash中。

[0063] 根据本发明提供的一种星载路由器键值存储的片外一次访存管理方法,当在所述第二布谷鸟哈希表中查找键值对时,计算待查找键值对在候选桶中的索引;在片内存储器的两个候选桶中确定与所述待查找键值对在候选桶中的索引对应的存储位置是否存在与所述待查找键值对匹配的指纹;确定与所述待查找键值对匹配的指纹存在,访问片外存储器的对应存储位置,查找键值对;或者确定与所述待查找键值对匹配的指纹不存在,返回键不存在的通知。

[0064] 具体地,为了查找一个流IDkey的值,检查其两个候选桶 $I_1[h_1(key)]$ 和 $I_2[h_2(key)]$ 。如果FP(key)存在于 $I_1[h_1(key)]$ 或 $I_2[h_2(key)]$,则发送一个读取(READ)请求从片外CuckooVault读取相应的键值对(key', value')。若key与key'匹配,则返回value'作为查找结果。否则,若在候选桶中未找到FP(key)或 $key \neq key'$,则返回查找失败。

[0065] 根据本发明提供的一种星载路由器键值存储的片外一次访存管理方法,当删除所述键值对时,所述方法还包括:计算待删除键值对在候选桶中的索引;在片内存储器的两个候选桶中确定与所述待删除键值对在候选桶中的索引对应的存储位置是否存在与所述待删除键值对匹配的指纹;确定与所述待删除键值对匹配的指纹存在,检查与所述待删除键值对匹配的指纹所在的槽位是否有锁;确定没有锁,从所述第一布谷鸟哈希表中删除与所述待删除键值对匹配的指纹。

[0066] 具体地,为了删除一个流IDkey,首先按照上述方法查找它。如果该key存在于片外CuckooVault中,则从片内CuckooIndex中删除其指纹。这里不需要从片外CuckooVault中删除其键值对,因为一旦从片内CuckooIndex中删除key的指纹,其在片外CuckooVault中的键值对最终将被其他元素覆盖。

[0067] 根据本发明提供的一种星载路由器键值存储的片外一次访存管理方法,当更新所述键值对时,具体包括:计算待更新键值对在候选桶中的位置;在片内存储器的两个候选桶中确定与所述待更新键值对在候选桶中的索引对应的存储位置是否存在与所述待更新键值对匹配的指纹;确定与所述待更新键值对匹配的指纹存在,检查与所述待更新键值对匹配的指纹所在的槽位是否有锁;确定没有锁,向与所述待更新键值对匹配的指纹所在的槽位添加写锁,然后通过写请求更新片内存储器和片外存储器中对应的槽位的值。

[0068] 为了更新一个流IDkey的值,首先按照上述方法查找它。若该流IDkey存在于片外CuckooVault中,则发送一个写入(WRITE)请求在片外CuckooVault中更新其值。

[0069] 在本发明的一种实施例中,对搜索到的最短踢出路径上的所有槽位执行原子性写

锁标记;若路径中存在已被锁定的槽位,则放弃当前路径并继续搜索下一候选路径;通过单次批处理读取操作获取踢出路径涉及的片外键值对;通过单次批处理写入操作完成键值对重新定位;迁移完成后原子性释放所有路径槽位的写锁。

[0070] 具体地,为了进一步加速星载路由器流表的操作,本发明在其片内CuckooIndex的槽上使用多线程结合互斥锁。首先描述插入过程。通过 BFS 找到替换路径后,本发明首先检查该路径上的每个槽,确保这些槽未被锁定。如果路径上有任何锁定的槽,本发明继续进行 BFS,直到找到完全未锁定的路径。此后,使用一个原子操作锁定该路径上的所有槽,然后执行两批批处理(READ和WRITE)请求来修改 CuckooVault。一旦写请求完成,使用一个原子操作解锁路径上的所有槽。对于具有插入/查找/删除/更新操作的混合工作负载,本发明可以使用多线程结合读和写(RW)锁来加速 CuckooDuo。

[0071] 对于查找,如果在 CuckooIndex 中找到匹配的指纹,本发明检查该槽是否有写锁。如果有,查找操作将被阻塞,直到写锁被释放。否则,相应的槽获取读锁,这意味着在其读锁被释放之前,该槽不能被插入/更新操作修改。

[0072] 对于插入,找到替换路径后,检查路径上的所有槽是否都没有锁。如果是,使用一个原子操作向路径上的所有槽添加写锁。否则,放弃该路径并继续搜索下一个最短路径。

[0073] 对于删除,在发现给定的键存在于 CuckooVault 中后,检查其在 CuckooIndex 中的槽是否没有锁。如果是,从 CuckooIndex 中删除指纹。否则,本发明等待其锁被释放并重新执行此操作。

[0074] 对于更新,在发现给定的键存在于 CuckooVault 中后,本发明也检查其在 CuckooIndex 中的槽是否没有锁。如果是,本发明向其槽添加写锁,并发送一个写(WRITE)请求来更新其值。写请求完成后,本发明释放写锁。

[0075] 根据本发明提供的一种星载路由器键值存储的片外一次访存管理方法,动态扩展所述候选桶,具体包括:将第一候选桶和第二候选桶按整数倍复制扩展;根据扩展倍数更新扩展候选桶的索引哈希函数;根据所述扩展候选桶的索引哈希函数清理扩展候选桶中的冗余元素;其中,清理冗余元素的模式包括主动模式和懒模式。

[0076] 具体地,当表已满时,键值存储需要扩展其大小以容纳更大的数据集。接下来,本发明首先介绍动态扩展一个 CuckooDuo 的基本方法,然后将本发明的方法与可扩展的扩展相结合,将全表扩展时间分摊到子表扩展中,并在插入过程中平滑空间利用率。

[0077] 1) 基本扩展方法。

[0078] CuckooDuo 的扩展基于内存复制,支持任意整数扩展比例 r 。要将 CuckooDuo 扩展 r 倍,首先执行复制操作,将 CuckooIndex 和 CuckooVault 的每个桶数组复制 $r-1$ 次,并将复制的桶附加到原始数组。例如,对于 CuckooIndex 的第一个数组 $I_1[0], \dots, I_1[m-1]$ 。内存复制后,它变为 $I_1[0], \dots, I_1[rm-1]$ 。其中,

$I_1[i], I_1[m+i], \dots, I_1[(r-1) \cdot m+i] (i \in \{0, 1, \dots, m-1\})$ 是相同的。接下来,本发明将用于选择候选桶的两个哈希函数从

$h_1(\text{key}) = H(\text{key}) \% m$ 和 $h_2(\text{key}) = (h_1(\text{key}) + \text{hash}(\text{FP}(\text{key}))) \% m$ 修改为

$h'_1(\text{key}) = H(\text{key}) \% (rm)$ 和 $h'_2(\text{key}) = (h_1(\text{key}) + \text{hash}(\text{FP}(\text{key}))) \% (rm)$ 。这样,内存复

制后,每个键仍然存在于其候选桶中,因为

$$h'_1(\text{key}) = H(\text{key}) \% (rm) \in \{H(\text{key}) \% m, H(\text{key}) \% m + m, \dots, H(\text{key}) \% m + (r-1)m\}$$

桶中存在冗余元素。CuckooDuo 还提供了两种扩展模式:主动模式和懒模式。

[0079] 根据本发明提供的一种星载路由器键值存储的片外一次访存管理方法,通过主动模式清理冗余元素,具体包括:在复制扩展候选桶后立即对所有扩展候选桶中的冗余元素进行删除;通过懒模式清理冗余元素,具体包括:在复制扩展候选桶后,为每个扩展候选桶设置待清理标记;当首次访问带标记的扩展候选桶时,将带标记的扩展候选桶中的所有冗余元素清除;将清楚冗余元素后的扩展候选桶的待清理标记删除。

[0080] 具体地,在主动模式下,CuckooDuo 在内存复制后立即检查所有桶以删除冗余元素。对于每个桶 $I_1[j]$,读取(READ)存储在 $B_1[j]$ 中的所有键,计算哈希函数以判断每个键是否应存储在 $I_1[j]$ 中,即检查 $h'_1(\text{key}) = j$ 是否成立。如果不成立,则从 CuckooIndex 中的 $I_1[j]$ 删除其指纹FP(key)。

[0081] 在懒模式下,CuckooDuo 在扩展后用 1 位指示符标记每个桶(存储在本地内存中)。之后,每次首次访问标记的桶 $I_1[j]$ 时,顺便读取并检查其中的所有元素,并像上述那样清理冗余元素。

[0082] 在本发明提供的一种实施例中,在片内存储器中维护扩展目录,记录每个子表CuckooDuo的地址范围及当前负载状态;所述扩展目录与所有CuckooIndex存储在快速内存中,所有CuckooVault存储在慢速内存中;当监测到任一子表CuckooDuo的负载超过阈值时:创建新的子表CuckooDuo,并通过内存复制操作将原子表拆分为两个子表;按扩展后的索引哈希函数 $h'_1 = H(\text{key}) \% (2m)$ 和 $h'_2 = (h'_1 + \text{hash}(\text{FP}(\text{key}))) \% (2m)$ 重新分配键值对;采用懒模式清理时,在目录中标记待清理子表,实际清理操作延迟至首次访问时执行;采用主动模式清理时,立即启动后台线程完成全量表扫描和冗余删除。

[0083] 2)可扩展的扩展方法。

[0084] 具体地,可扩展的扩展(Extendible Expansion)通过维护一个目录来工作,该目录将每个元素索引到一个子表中。当一个子表已满时,通过创建一个新的子表并将其中将近一半的元素移动到新子表中将其拆分。CuckooDuo的扩展过程也可以采用该方式工作。具体而言,本发明用CuckooDuo替换上述的子表。目录以及所有 CuckooIndexes 存储在快速内存中,所有 CuckooVaults 存储在慢速内存中。每当一个 CuckooDuo(子表)已满时,通过执行如基本扩展中所述的内存复制操作将其拆分为两个 CuckooDuos。之后,可以立即读取所有桶以清理冗余元素(主动模式),或者在访问桶时顺便清理冗余元素(懒模式)。

[0085] 为了使得本发明的目的,技术方案以及优点更加清楚明白,以下结合附图当中的实例对本发明进行更进一步详细说明。

[0086] 图2是本发明将元素以直接方式插入CuckooDuo流表的过程。假设 $\text{hash}(\text{FP}) = \text{FP}$ 且 $m = 3, d = 2, L = 1$ 。为了插入一个元素 $e_1 = \langle \text{key}_1, \text{value}_1 \rangle$,首先计算哈希函数去定位到CuckooIndex的两个候选桶中 $I_1[h_1(\text{key}_1)]$ 和 $I_2[h_2(\text{key}_2)]$ 中。由于在 $I_1[h_1(\text{key}_1)]$ 中正好还有一个空槽,本发明直接将该元素插入到该空槽,即将FP(key_1)写入CuckooIndex的一

个空槽位,并将 e_1 插入到CuckooVault中的相应槽位。

[0087] 图3是本发明将元素以BFS方式插入CuckooDuo流表的过程。假设 $hash(FP) = FP$ 且 $m = 3, d = 2, L = 1$ 。为了插入一个元素 $e_2 = \langle key_2, value_2 \rangle$,首先计算哈希函数去定位到CuckooIndex的两个候选桶中。由于 key_2 的两个候选桶都已经满了,所以通过将候选桶中的所有指纹入队来启动BFS。然后,逐个出队指纹。对于第一个出队的指纹 $FP(key_3) = 0011$,计算 $h_2(key_3) = (h_1(key_3) + hash(FP(key_3))) \% m = (1 + 3) \% 3 = 1$,由于备用桶 $I_2[1]$ 也已满,继续出队 $FP(key_1) = 0001$ 并计算

$h_2(key_1) = (h_1(key_1) + hash(FP(key_1))) \% m = (1 + 1) \% 3 = 2$ 。由于 $I_2[2]$ 未滿,本发明找到了最短的踢出路径。从CuckooVault中读取(READ) $e_1 = \langle key_1, value_1 \rangle$,将 e_1 和 e_2 写入(WRITE)它们的新位置,并最终更新指纹。

[0088] 图4是本发明提出的双指纹优化处理指纹冲突的过程。当插入key时,发现其指纹已经存在于 $I_1[h_1(key_1)]$ 的一个主槽位中。读取 $I_1[h_1(key_1)]$ 中的所有键,确定key与 key_3 发生了 FP_1 冲突。然后尝试通过在 $I_1[h_1(key_1)]$ 中调整指纹来解决冲突。在确保主槽位中的键与key不存在 FP_2 冲突的前提下,尝试将 $FP_2(key_1)$ 插入备份槽位。计算 $FP_2(key_2)$, $FP_2(key_3)$ 和 $FP_2(key_4)$,确保它们与 $FP_2(key_1)$ 不冲突。最后,将移动后的元素写入它们的目标位置。

[0089] 图5是本发明将CuckooDuo流表扩展3倍的过程。为了清晰起见,仅考虑第一个桶数组 I_1/B_1 ,其中 $m = 2$ 。扩展操作通过将 I_1/B_1 复制2次来实现,导致每个数组中有 $r \times m = 6$ 个桶。之后,在主动模式下清理 $I_1[1]$ 中的冗余元素(或在懒模式下首次访问时),读取(READ)其中的所有键并检查每个键是否应存储在此处。对于 key_2 ,由于 $h'_1(key_2) = 9 \% 6 = 3$,其候选桶变为 $I_1[3]$,所以从 $I_1[1]$ 中删除其指纹。如前所述,不需要删除远程CuckooVault中的实际元素,它最终将被后续插入的元素覆盖。

[0090] 本发明提出的“片内-片外协同星载路由器流表动态完美哈希结构”具备显著的技术效果,特别适用于资源受限、高延迟、高带宽敏感的星载路由器环境。通过构建CuckooIndex与CuckooVault之间的一一映射关系,实现了键值对查找与更新操作仅需一次片外访存,显著降低了远程访问延迟和带宽消耗,有效克服了现有完美哈希方案负载率低、扩展代价高、无法识别非法访问等核心瓶颈。

[0091] 本发明方案具备如下主要技术优势:

首先,所提出的哈希结构具备关联性为1的完美索引能力,每个键值对均通过片内索引精确指向唯一片外槽位,从而使得查找和更新操作无需访问多个候选桶,有效消除多路径带来的冗余访存行为,极大提升了远程存取效率。

[0092] 其次,通过引入双指纹优化机制(Dual-Fingerprint),有效缓解了传统指纹结构中的哈希冲突问题,在保持指纹长度不变的前提下显著提升了系统负载率,在实验条件下可达到超过97%的装载率,同时仅需极小的辅助stash存储,即可避免指纹冲突带来的插入

失败与性能劣化。

[0093] 再次,本发明通过在本地索引中引入广度优先搜索(BFS)最短kick-out路径发现机制,可确保每次插入过程最少数据迁移路径,从而在高负载条件下仍保持较低插入延迟。该机制使得在大多数情况下插入操作仅需一次写操作,在最坏情况下也仅需一次读+一次写(2个RTT),大幅优于现有需要多次往返访问的哈希迁移方案。

[0094] 此外,本发明提出的非幂次容量支持及懒/主动模式扩展策略,成功打破了传统CuckooFilter对容量大小的2的幂限制,允许任意表大小的构建与线性扩展,显著提高了内存利用率,并通过本地删除指纹的方式完成扩容过程中冗余项清理,避免了传统方案中大规模数据迁移的开销。实验结果表明,该扩展方式可实现2倍扩展加速,懒模式下扩展时间降低约6.67倍。

[0095] 最后,本发明支持在本地结构中快速识别非存在键,有效过滤非法查找、非法插入和冗余更新等操作,非法访问延迟降低20倍以上,进一步节省了远程通信资源,是当前星载系统中极为重要的性能保障机制。

[0096] 综上所述,本发明在结构设计、访问路径优化、动态扩展、冲突处理、带宽控制等方面均取得突破,具备高负载、高速、低延迟、低功耗、强动态性与可扩展性等关键技术效果,显著优于现有技术方案,特别适用于星载路由器、低轨卫星计算平台、异构高性能存储节点等片内-片外协同处理场景。

[0097] 下面对本发明提供的星载路由器键值存储的片外一次访存管理装置进行描述,下文描述的星载路由器键值存储的片外一次访存管理装置与上文描述的星载路由器键值存储的片外一次访存管理方法可相互对应参照。

[0098] 如图6所示为本发明提供的一种星载路由器键值存储的片外一次访存管理装置,包括:

第一部署模块610,用于在星载路由器的片内存储器中部署第一布谷鸟哈希表;

第二部署模块620,用于在星载路由器的片外存储器中部署第二布谷鸟哈希表;

其中,所述第一布谷鸟哈希表用于存储键的指纹;所述第二布谷鸟哈希表用于存储完整的键值对;所述第一布谷鸟哈希表中的每个指纹与所述第二布谷鸟哈希表中同一索引位置的键值对一一对应。

[0099] 图7示例了一种电子设备的实体结构示意图,如图7所示,该电子设备可以包括:处理器(processor)710、通信接口(Communications Interface)720、存储器(memory)730和通信总线740,其中,处理器710,通信接口720,存储器730通过通信总线740完成相互间的通信。处理器710可以调用存储器730中的逻辑指令,以执行星载路由器键值存储的片外一次访存管理方法,该方法包括:在星载路由器的片内存储器中部署第一布谷鸟哈希表;在星载路由器的片外存储器中部署第二布谷鸟哈希表;其中,所述第一布谷鸟哈希表用于存储键的指纹;所述第二布谷鸟哈希表用于存储完整的键值对;所述第一布谷鸟哈希表中的每个指纹与所述第二布谷鸟哈希表中同一索引位置的键值对一一对应。

[0100] 此外,上述的存储器730中的逻辑指令可以通过软件功能单元的形式实现并作为独立的产品销售或使用,可以存储在一个计算机可读取存储介质中。基于这样的理解,本发明的技术方案本质上或者说对现有技术做出贡献的部分或者该技术方案的部分可以以软件产品的形式体现出来,该计算机软件产品存储在一个存储介质中,包括若干指令用以

使得一台计算机设备(可以是个人计算机,服务器,或者网络设备等)执行本发明各个实施例所述方法的全部或部分步骤。而前述的存储介质包括:U盘、移动硬盘、只读存储器(ROM, Read-Only Memory)、随机存取存储器(RAM, Random Access Memory)、磁碟或者光盘等各种可以存储程序代码的介质。

[0101] 另一方面,本发明还提供一种计算机程序产品,所述计算机程序产品包括计算机程序,计算机程序可存储在非暂态计算机可读存储介质上,所述计算机程序被处理器执行时,计算机能够执行上述各方法所提供的星载路由器键值存储的片外一次访存管理方法,该方法包括:在星载路由器的片内存储器中部署第一布谷鸟哈希表;在星载路由器的片外存储器中部署第二布谷鸟哈希表;其中,所述第一布谷鸟哈希表用于存储键的指纹;所述第二布谷鸟哈希表用于存储完整的键值对;所述第一布谷鸟哈希表中的每个指纹与所述第二布谷鸟哈希表中同一索引位置的键值对一一对应。

[0102] 又一方面,本发明还提供一种非暂态计算机可读存储介质,其上存储有计算机程序,该计算机程序被处理器执行时实现以执行上述各方法提供的星载路由器键值存储的片外一次访存管理方法,该方法包括:在星载路由器的片内存储器中部署第一布谷鸟哈希表;在星载路由器的片外存储器中部署第二布谷鸟哈希表;其中,所述第一布谷鸟哈希表用于存储键的指纹;所述第二布谷鸟哈希表用于存储完整的键值对;所述第一布谷鸟哈希表中的每个指纹与所述第二布谷鸟哈希表中同一索引位置的键值对一一对应。

[0103] 以上所描述的装置实施例仅仅是示意性的,其中所述作为分离部件说明的单元可以是或者也可以不是物理上分开的,作为单元显示的部件可以是或者也可以不是物理单元,即可以位于一个地方,或者也可以分布到多个网络单元上。可以根据实际的需要选择其中的部分或者全部模块来实现本实施例方案的目的。本领域普通技术人员在不付出创造性的劳动的情况下,即可以理解并实施。

[0104] 通过以上的实施方式的描述,本领域的技术人员可以清楚地了解到各实施方式可借助软件加必需的通用硬件平台的方式来实现,当然也可以通过硬件。基于这样的理解,上述技术方案本质上或者说对现有技术做出贡献的部分可以以软件产品的形式体现出来,该计算机软件产品可以存储在计算机可读存储介质中,如ROM/RAM、磁碟、光盘等,包括若干指令用以使得一台计算机设备(可以是个人计算机,服务器,或者网络设备等)执行各个实施例或者实施例的某些部分所述的方法。

[0105] 最后应说明的是:以上实施例仅用以说明本发明的技术方案,而非对其限制;尽管参照前述实施例对本发明进行了详细的说明,本领域的普通技术人员应当理解:其依然可以对前述各实施例所记载的技术方案进行修改,或者对其中部分技术特征进行等同替换;而这些修改或者替换,并不使相应技术方案的本质脱离本发明各实施例技术方案的精神和范围。

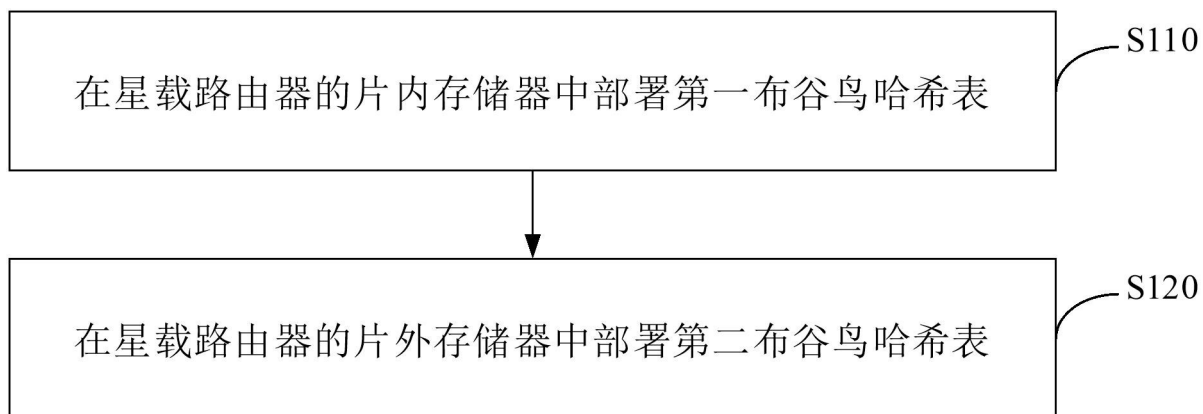


图1

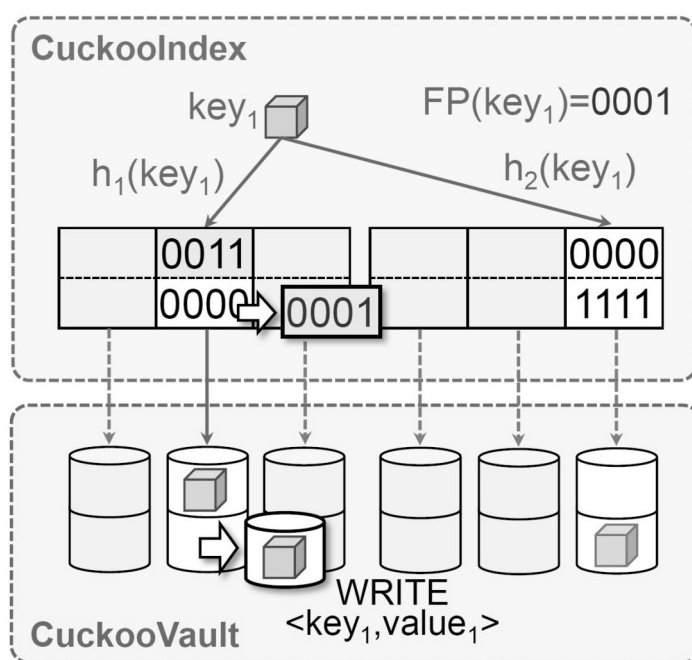


图2

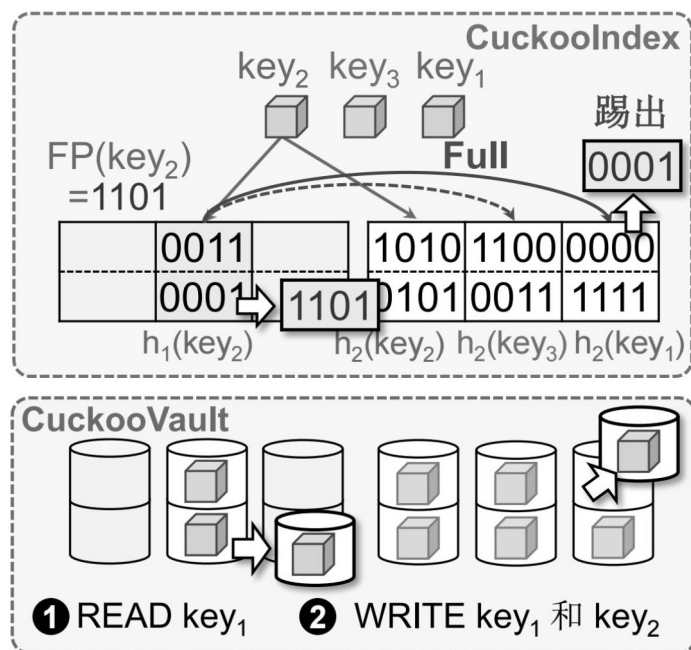


图3

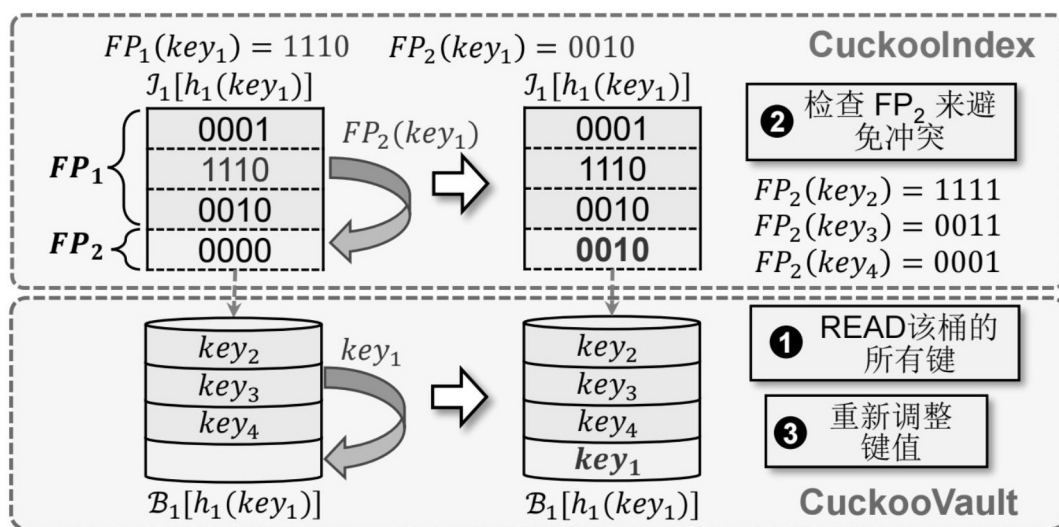


图4

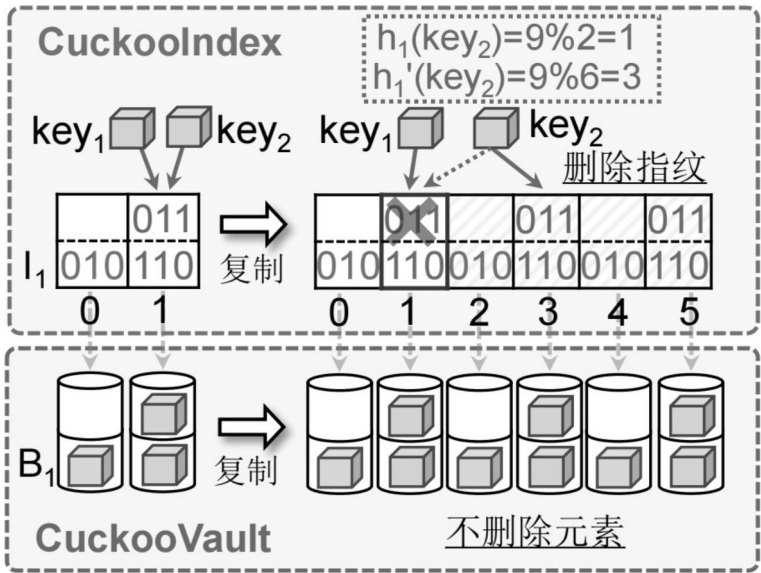


图5

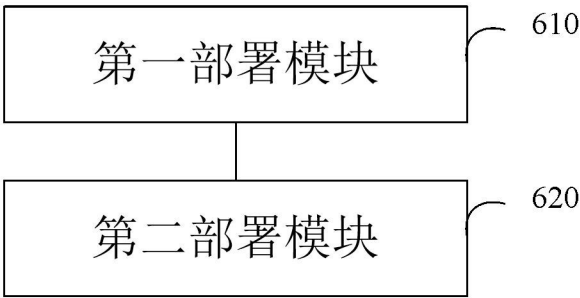


图6

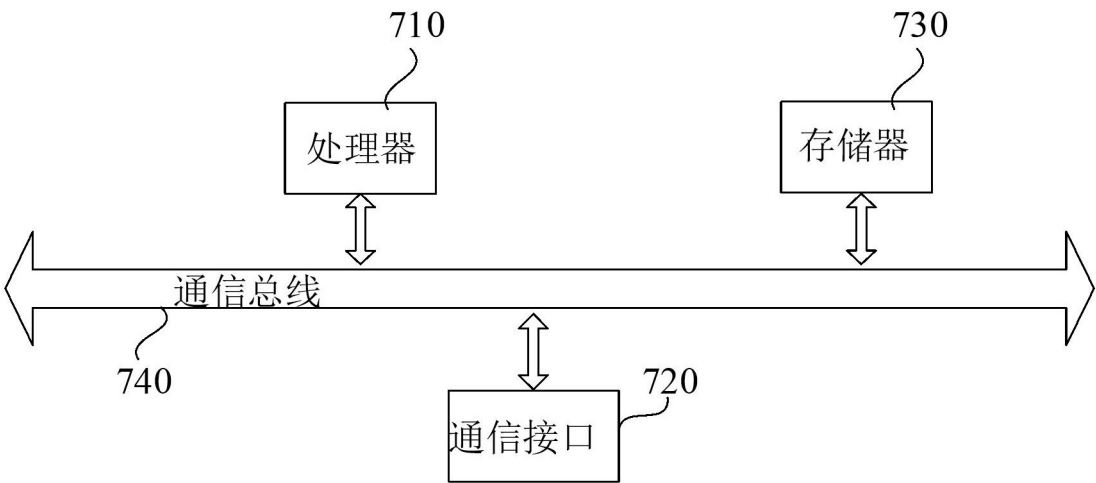


图7